

Websockify as the proxy and noVNC for rendering on websites. The following commands can be added to the cell:

```
!echo "deb [trusted=yes] https://packages.mozilla.org/apt/mozilla main" | sudo tee -a /etc/apt/sources.list.d/mozilla.list
!sudo apt-get update
!printf "49\n1\n17\n" | apt-get install -y xfce4 xfce4-goodies tightvncserver novnc websockify firefox/mozilla
```

Next, set a password for the VNC server for the root user. The `vncpasswd` command is used for this. A default password 'password' can be set as:

```
!printf "password\npassword\n\n\n" | vncpasswd
```

Now start the VNC server for display 1.

```
!sudo vncserver
!export display=:1
```

It runs on port 5091 by default for display 1. Websockify

```
!printf "password\npassword\n\n\n" | vncpasswd
!sudo vncserver
!export display=:1
!websockify -D --web=/usr/share/novnc/ 6080 localhost:5901
```

Figure 1: Setting up the VNC server

```
!ifconfig

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.28.0.12 netmask 255.255.0.0 broadcast 172.28.255.255
    ether 02:42:ac:1c:00:0c txqueuelen 0 (Ethernet)
    RX packets 123 bytes 32073 (32.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 84 bytes 21147 (21.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 229 bytes 54045 (54.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 229 bytes 54045 (54.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 2: Checking IP addresses

```
[ ] !npm install -g localtunnel
import subprocess
process = subprocess.Popen('lt --port 6080'.split(), stdout=subprocess.PIPE, stderr=subprocess.PIPE, bufsize=0)
output = process.stdout.readline().decode('utf-8').strip()+'/vnc.html'
url = output[output.find('https'):]
```

Figure 3: Running Localtunnel

```
import requests
ltpassword=requests.get("https://loca.lt/mytunnelpassword").text
```

Figure 4: Getting the local tunnel password

is used to proxy it to port 6080 (or any other port).

```
!websockify -D --web=/usr/share/novnc/ 6080 localhost:5901
```

This port is to be exposed to the public internet. Since Colab runs on a container, it is likely that it is behind a NAT and is not routable over the internet. This is confirmed by checking the IP addresses of the container by running `ifconfig` (Figure 2).

There are a few options for NAT (network address translation) traversal and exposing the port to the internet, with Ngrok and Localtunnel being prominent ones. Ngrok is a good one, but it has recently added credit card verification. Hence, we are going to use Localtunnel. It can be installed from 'npm' using the following command:

```
!npm install -g localtunnel
```

It is run by a Python subprocess so that we can take out the hosted URL easily:

```
import subprocess
process = subprocess.Popen('lt --port 6080'.split(),
    stdout=subprocess.PIPE, stderr=subprocess.PIPE,
    bufsize=0)
output = process.stdout.readline().
    decode('utf-8').strip()+'/vnc.html'
url = output[output.find('https'):]
```

Unlike Ngrok, Localtunnel needs a password to be opened. The password can be fetched using an `http` request. Python does this by using the `requests` library.

```
import requests
ltpassword=requests.get('https://loca.lt/mytunnelpassword').text
```

Ultimately, it is rendered as an `iframe` from Colab after clearing the current output.

```
from IPython import display
display.clear_output()
display.HTML(f'<h1>LT Password: {ltpassword}</h1><br><h1>VNC Password: {vncpassword}</h1><br><iframe style="border: 0; width: 100%; aspect-ratio: 16/9; overflow: auto;" src="{url}"></iframe>')
```

The complete code in the cell can be summarised as:

```
!echo "deb [trusted=yes] https://packages.
```